Smaller Gate Counts for the AES S-box and Inverse S-box in the AND/XOR/XNOR Model:

106 gates at depth 24, 109 gates for the inverse, and 121 gates at depth 16

Swapnil Jain

Preprint, October 2026

Abstract

We report AES S-box and inverse S-box circuits with fewer gates than the best circuits we could find in a limited literature check (not a systematic survey), in the model where AND, XOR and XNOR gates cost one each. At depth 16 we give 121 gates. The smallest published circuit of depth at most 16 that we found has 125 gates; we recounted its depth as 15 in the NIST file and 16 in the CMT file, so our 121 is one level deeper than the first and as deep as the second. At depths 17, 18 and 21 we give 116, 112 and 111 gates; these are Pareto points between the published 125/15 and 110/22, not improvements on a nearby circuit (the 111/21 is +1 gate and −1 level against the NIST 110/22). The 112/20 is also released but dominated. Each depth-budget circuit shares 72% to 86% of its gate functions with its closest published circuit, against 56% to 74% between published circuits that are not siblings. The 121/16 is most likely a −4-gate local improvement of the CMT 125/16 (86% shared), and the 111/21 most likely a rebalancing of the NIST 110/22 (83%, the level of the NIST sibling pair). The 112/18, 112/20, 111/21 and inverse 109/30 are not under our hash commitment, and both commitment roots postdate the 2026 NIST additions. For size alone we give 106 gates at depth 24 (also 106 gates at depth 34 with 32 ANDs), against 110 gates at depth 22 on the NIST page and 113 gates at depth 27 on the CMT page. For the inverse S-box we give 109 gates at depth 30, against 112 at depth 26 (NIST) and 121 at depth 21 (CMT page); it shares only 22% to 23% of its gate functions with either, far below the 64% the two prior inverse circuits share, so it is not a local edit of either. Both size circuits are deeper than the circuits they beat in size. The AND count (32 to 34) is about that of earlier work, so we claim no gain for masked implementations. The 106/34 comes from a *representation sweep*: rebuilding the Boyar–Peralta layout in each of 244,800 field representations, then running a gate-preserving random walk. Scaling and Frobenius transformations of the linear matrices were tried before by Maximov and Ekdahl (2040 variants), and similar work is due to Ueno et al. and, for representation choice, Canright; what we add is a wider sweep combined with the walks, not the idea of varying the representation. Circuits in richer gate sets (Maximov and Ekdahl's 102 gates with NAND, NOR and MUX; the lower-depth SLICE circuits) were not translated into our model, so our comparisons hold only in the {AND, XOR, XNOR} model. Every circuit is checked exhaustively over 256 inputs, symbolically (which carries the same information) and against checker-sanity controls, and has a Lean 4 kernel proof of equality with a literal table; that the table is the S-box rests on the Python verifier. We make no optimality claim.

Keywords: AES S-box, gate count, circuit minimization, bitslicing, formal verification.

1 Introduction

The AES S-box is the inversion in $\mathbb{F}_{2^8}$ (modulus $x^8 + x^4 + x^3 + x + 1$) followed by an affine map [14]. Its implementation cost is a standard target for circuit minimization, because it dominates the nonlinear cost of AES in hardware and in bitsliced software. In this paper cost means the number of two-input gates in the basis {AND, XOR, XNOR}, each counted as one (Section 2).

Prior work we build on. Canright's tower-field construction [8] reduces the inversion to arithmetic in $\mathbb{F}_{2^4}$ and $\mathbb{F}_{2^2}$. Boyar and Peralta [4, 5] turned this into a circuit with a linear top layer, a small nonlinear middle and a linear bottom layer, and minimized the linear layers with a distance-based heuristic. Their published circuit has 115 gates at depth 28 [5]; the version improved by Calik, with 113 gates (32 AND) at depth 27, is listed on the Circuit Minimization Team (CMT) page [1]. The same group gave depth-constrained circuits [6, 7]; the CMT page lists a circuit with 125 gates at depth 16 [7]. Linear layers are costed with the heuristics of Paar [15] and Boyar and Peralta [4, 5], including the randomised variant RNBP [16]. Maximov and Ekdahl [9] reach 102 gates for the forward and 101 (plus one NOT gate) for the inverse S-box in a richer gate set, and a 111-gate forward circuit of depth 14. Reyhani-Masoleh, Taha and Ashmawy [10] give S-box implementations in a richer gate library (as listed by Maximov and Ekdahl: 119 gates at depth 20 and 129 at depth 16, each plus inverters, with NOR gates), and Ueno et al. give low-depth circuits (as listed there: 151 at depth 15 [11] and 149 at depth 15 [12], each plus inverters); we did not compare these gate by gate. The SLICE search of Kim, Baek, Jeon, Hadžić and Kim [13] lists, in its artifact, AES S-box circuits of 128 gates at depth 14 (33 nonlinear and 95 linear), 136 at depth 13 and 149 at depth 12, using NAND, NOR, AND, OR, XOR and XNOR; they are lower-depth and larger than ours and in a richer gate set, so they do not affect our claims at depth 16 and above. In 2026 the NIST circuit-complexity page [2] (forward circuits) and its data repository [3] (forward and inverse circuits) added S-box circuits over {AND, XOR, XNOR}, among them 110 gates at depth 22 for the forward and 112 gates at depth 26 for the inverse S-box, which is where the published numbers stood when we compared.

Prior art for varying the representation. Varying the field representation is not new. Canright [8] searched over the choice of basis for the most compact tower-field S-box. Maximov and Ekdahl [9] (Section 5.4, "Additional Transformation Matrices") write the S-box as $\mathrm{SBox}(x) = x^{-1} \cdot A$ and observe that, because scaling by a constant α and the Frobenius power $x \mapsto x^{2^\beta}$ are linear, any choice $\alpha \in \{1, \ldots, 255\}$, $\beta \in \{0, \ldots, 7\}$ gives new top and bottom matrices around the same inversion block; they test all 2040 variants and keep the best, for the forward and the inverse S-box, in both of their architectures, and (Section 5.4.1) independently on the forward and inverse parts of the combined S-box (2040^2 variants). They report that Ueno et al. [12] independently considered a similar approach but only with multiplication by a constant, not squaring. Our sweep (Section 3) contains the same scaling and Frobenius parameters (t, g); this part is not new. What differs is that we also vary the dual basis (θ_0, θ_1) of $\mathbb{F}_{2^8}$ over $\mathbb{F}_{2^4}$, giving 244,800 builds, we rebuild the whole Boyar–Peralta layout in each representation in the {AND, XOR, XNOR} model rather than re-optimizing the matrices around one fixed inversion block, and we follow the sweep with gate-preserving walks. We did not test whether the Maximov–Ekdahl variants reach any of our representations, and we have no evidence that the dual-basis part is what gains the gates. The reported counts of Maximov

and Ekdahl are in a gate set with NAND, NOR and MUX; we did not translate their circuits into {AND, XOR, XNOR}.

Contributions.

- Forward S-box circuits with fewer gates than the smallest published circuit whose depth is at most ours, which has 125 gates and depth 15 (ours are one to six levels deeper than it): 121 at depth 16, 116 at 17, 112 at 18 and 111 at 21 (Table 1). The last three are Pareto points between the published 125/15 and 110/22; the 111/21 is +1 gate and −1 level against the NIST 110/22. They share the representation of published circuits, and the 121/16 and 111/21 are most likely local edits of the CMT 125/16 and the NIST 110/22 (Section 4.3). Two further circuits, 113 at depth 18 and 112 at depth 20, are released but dominated by our 112 at depth 18.
- The smallest circuits in this gate model that we found, with a depth trade-off: 106 gates at depth 24 (and at depth 34, dominated by the first) for the forward and 109 gates at depth 30 for the inverse S-box. These are deeper than the circuits they beat in size.
- A wider representation sweep, combined with gate-preserving walks, that reaches 106 where walks inside one representation stopped at 110 (Section 3). The idea of scaling and Frobenius variants is that of Maximov and Ekdahl; see "Prior art for varying the representation".
- A verification pipeline ending in Lean 4 kernel proofs, a commitment of circuit hashes, and a self-contained Python verifier.
- An account of what did not work within our method (Section 5).

What we do not claim. We make no optimality claim; every count is the best we found. We claim no improvement on the 110-gate depth-22 circuit in the sense of a tie or a dominated point. Our 106 is smaller but deeper. Circuits in richer gate sets (for example Maximov and Ekdahl's) are smaller at these depths, so the depth-budget comparisons hold only in the {AND, XOR, XNOR} model. Our literature check covers the CMT page, the NIST page and repository file listings, the S-box tables of Maximov and Ekdahl, the artifact listing of SLICE [13], and the sources in the bibliography; it is not a systematic survey. We did not run technology mapping, so we make no area or timing claim. Maximov and Ekdahl's circuits use richer gates; a translation of them into {AND, XOR, XNOR} was not attempted, so we make no comparison with them.

1.1 Results at a glance

Verification level **L** = exhaustive over 256 inputs + symbolic + checker-sanity controls + Lean 4 kernel proof of equality with a literal table; all eleven released circuits are L. Depth counts every gate as one level. The comparison column is the smallest published circuit in the same model with depth at most ours (for the depth-budget rows) or the smallest published circuit (for the size rows); for the depth-budget rows we add the nearest published neighbour, the NIST 110/22. We recounted the depth of the prior circuits we quote, from the files, with the depth function of our verifier (see the paragraph after Table 1). The "smallest published with depth at most d" statements rest on the CMT page [1] and the 34 forward and 25 inverse circuit files in the NIST repository [3] (the forward directory lists 35 entries,

Ours (g / d)	A+X+XN	AD	Published, smallest with depth $\leq d$ (g / d, AD)	Δ (g, d)	Comparison
Forward S-box, Pareto points between the published 125/15 *and* 110/22					
121 / 16	$34 + 83 + 4$	4	125 / 15, 4 [3]	$-4, +1$	smaller, one level deeper than the NIST file
			125 / 16, 4 [1]	$-4, 0$	smaller, same depth as the CMT file
116 / 17	$34 + 78 + 4$	4	125 / 15, 4	$-9, +2$	gap in the published front
			110 / 22, 4/5 [3]	$+6, -5$	larger, shallower
112 / 18	$34 + 75 + 3$	4	125 / 15, 4	$-13, +3$	gap in the published front
			110 / 22	$+2, -4$	larger, shallower
111 / 21	$34 + 74 + 3$	4	125 / 15, 4	$-14, +6$	gap in the published front, not a 14-gate improvement (Section 4.3)
			110 / 22	$+1, -1$	**one gate larger, one level shallower**
Forward S-box, smallest size we found, with a depth trade-off					
106 / 24	$34 + 69 + 3$	4	110 / 22, 4/5 [2, 3]	$-4, +2$	smaller, **deeper**
			113 / 27, 6 [1]	$-7, -3$	smaller and shallower (the 110/22 dominates the 113/27)
Inverse S-box, smallest size we found, with a depth trade-off					
109 / 30	$32 + 71 + 6$	6	112 / 26, 5 [3]	$-3, +4$	smaller, **deeper**
			121 / 21, 4 [1]	$-12, +9$	smaller, **deeper**
Released but dominated by another circuit of ours (not claimed)					
113 / 18	$34 + 76 + 3$	4	125 / 15, 4	$-12, +3$	dominated by our 112/18
112 / 20	$34 + 75 + 3$	4	125 / 15, 4	$-13, +5$	dominated by our 112/18
106 / 34	$32 + 71 + 3$	6	110 / 22, 4/5	$-4, +12$	dominated by our 106/24; the sweep circuit
109 / 31	$32 + 70 + 7$	6	112 / 26, 5	$-3, +5$	dominated by our 109/30

Table 1: Results; all circuits have verification level L. A+X+XN = AND+XOR+XNOR counts; Δ = ours minus published (gates, depth); AD is the AND depth (most AND gates on a path), computed for ours and for the published circuits with `tools/and_depth.py`; the two NIST 110/22 files have AD 4 and 5. The 106 at depth 24 and the 109 at depth 30 are deeper than the circuits (110/22, 112/26) that they beat in size; we do not call either better in both measures. Our earlier 110-gate circuit at depth 38 ($32 + 73 + 5$, L) is dominated by the NIST 110/22 and is not claimed; a 110/22 circuit that we also held turned out to be identical to the NIST file and is not ours (Section 4.3). Maximov–Ekdahl [9] points (Tables 4 and 8 there), in gates / depth, in a gate set with NAND, NOR and MUX; "MI" is their notation. Forward: 130/12 (77 XOR, 1 XNOR, 4 AND, 37 NAND, 5 NOR, 6 MUX), 111/14 (61 XOR, 8 XNOR, 27 NAND, 5 NOR, 8 MUX, 2 MI), 102/24 (58 XOR, 6 XNOR, 27 NAND, 5 NOR, 6 MUX). Inverse: 130/12 (68 XOR, 10 XNOR, 41 NAND, 5 NOR, 6 MUX), 110 plus one NOT/14 (64 XOR, 4 XNOR, 27 NAND, 5 NOR, 8 MUX, 2 MI), 101 plus one NOT/25 (56 XOR, 7 XNOR, 27 NAND, 5 NOR, 6 MUX). A translation of these circuits into {AND, XOR, XNOR} was not attempted, so they are not compared with ours.

one of which is `README.md`), whose names encode the gate count and depth. The smallest listed forward circuit with depth at most 21 has 125 gates; the smallest listed inverse circuit with depth at most 30 has 112 gates. We parsed the circuits we quote with our own code (`tools/prior_circuits.py`; the files are used as data only), checked that each computes the S-box or inverse S-box, and recounted the depth with the depth function of our verifier. The NIST file `aes-sbox-a34-ad4-g125-gd15-xx91-4` has 125 gates ($34 + 87 + 4$) and depth 15, as its name says. The CMT file `AESDEPTH16SIZE125` has 125 gates ($34+87+4$) and depth 16, as the CMT page says. They are two different circuits (they share 88 of 125 gate functions). Both recounts agree with the stated values for the other files we quote: the NIST 110/22 circuits (two files, depth 22), the NIST inverse 112 (depth 26), the CMT 113 (depth 27) and the CMT inverse 121 (depth 21). Because the depth-15 circuit exists, we do not call our 121 at depth 16 a same-depth win: it is one level deeper than the NIST 125 and as deep as the CMT 125.

2 Gate model and verification

2.1 Gate model

A circuit is a straight-line program over two-input gates AND, XOR and XNOR, each of cost 1, with free fan-out. $\mathrm{XNOR}(x, y) = 1 \oplus x \oplus y$. There is no separate NOT gate and no constant input, so every negation is absorbed in an XNOR. Bits are *MSB-first*: inputs $U_0, \ldots, U_7$ with U_0 the most significant bit of the input byte, and outputs $S_0, \ldots, S_7$ with S_0 the most significant bit of the output byte. The forward circuit maps the byte x to the FIPS-197 S-box value [14]; the inverse circuit maps y to the inverse permutation. Depth is the number of gates on the longest path, with every gate counting one. We count totals as AND + XOR + XNOR and always list the three counts. The gate model is the one of the CMT 113-gate circuit and, for the 2026 circuits, the model the NIST page states.

2.2 Verification

Each circuit is checked by a program (`verify_sbox.py` in the pack) written to depend on nothing else; it needs only Python 3. The table is computed inside the program from inversion in $\mathbb{F}_{2^8}$ and the affine map, and its inverse permutation gives the inverse table. The program asserts the FIPS-197 values $S(0) = \texttt{63}$, $S(1) = \texttt{7C}$, $S(\texttt{53}) = \texttt{ED}$ and $S^{-1}(\texttt{63}) = 0$.

1. **Exhaustive.** All 256 inputs are evaluated bit-sliced (each signal is a 256-bit integer) and compared with the table.

2. **Symbolic.** Each signal is computed as its exact algebraic normal form over $\mathbb{F}_2$ (a set of monomials over $U_0, \ldots, U_7$), and each output must equal the normal form of the table, obtained by a Möbius transform. Over 8 variables this carries the same information as the exhaustive test; it is a different evaluation path, but it uses the same reference table and the same circuit parsing as the exhaustive check.

3. **Must-fail controls (checker-sanity tests).** The program applies seven mutations that must be rejected: AND→XOR, XOR→AND, XOR→XNOR, a rewired operand, two swapped outputs, a reversed input order, and a table with one entry (`0x53`) changed. These show that the checker can reject; they do not test the circuit independently. The table-entry mutation cannot fail for a circuit that has passed the exhaustive check, and the swapped-output and reversed-input mutations test only the comparison, so none of

the three is evidence about the circuit. All seven are rejected for every released circuit. The search code also has its own checker (`tools/sbox.py check`, with four controls) and a second independent evaluator for the 106-gate circuit (`ideas/sbox/indep_check.py`, with its own log tables, affine map and evaluator), both of which pass; they are not part of the pack.

4. **Lean 4 kernel proof.** The theorem, as written in each Lean file, is

```
theorem circuit_ok :  allOk gates sbox = true := by decide +kernel
```

where `sbox` is the literal 256-entry table (the inverse table for the inverse circuits), `gates` is the gate list with signals 0 to 7 the inputs, and `allOk gs t` is `(List.range 256).all fun x => evalPt gs x == t.getD x 0`. By unfolding the definitions this is equivalent to the statement that the circuit equals the table at every $x < 256$; the files also contain `circuit_pointwise :` $\forall x, x < 256 \rightarrow$ `evalPt gates x = sbox.getD x 0`, derived from `circuit_ok`. The file also proves two controls *false*: `control_mutant` (one AND gate turned into an XOR) and `control_bad_table` (the table with one entry changed). Both are checker-sanity tests; `control_bad_table` follows logically from `circuit_ok` and cannot fail once that holds. From the saved outputs, `#print axioms` lists `[propext]` for `circuit_ok` and for both controls, and `[propext, Quot.sound]` for `circuit_pointwise`. We use core Lean without Mathlib. The Lean theorem is equality with a literal table; that the table is the AES S-box rests on the Python verifier, which computes it from field inversion, and gate counts and depths are not Lean-checked. The Lean files were generated by `gen_sbox_lean.py` from the circuit files, and are not themselves covered by the hash commitment. We regenerated the 106-gate depth-24 file with the generator in the pack: its gate list and theorem match the released file, and only a comment line in the header differs (the source path), so the released files were written by a slightly earlier version of the generator; for the others we compared the gate lists with the JSON files.

All eleven files were checked together, in their released form, on one machine with Lean 4.34.1 (x86_64 Linux release, recorded in `lean/lean_version.txt`); each `.out` file records the exit status (0) and a wall-clock time between 8.3 and 9.2 s. For all eleven released circuits, `circuit_ok` depends only on `propext`, and `circuit_pointwise` on `propext` and `Quot.sound`. The pack's `lean/SHA256SUMS` lists the hashes of the Lean files and outputs, which are not part of the commitment.

2.3 Commitment

We recorded SHA-256 hashes of the circuit files in a file `SHA256SUMS`, in two rounds, both published in GitHub gists dated 2026-10-11. The first root, `f9cfddf4...`, is in the gist `3b2c5d8c...`; the first-round file is in the pack as `commitment/SHA256SUMS.v1` and hashes to that root. The second root, which supersedes it, is

```
7139c329b0b76fe96c95e9012c4523c2ca5220a41bde366bc0dc0e800efaf078,
```

in the gist `https://gist.github.com/Swapnil-jain/6750e3f460658bc51c9a5f376c93af72`; it added the 107-gate depth-13 and 108-gate depth-12 circuits of the companion multiplier paper, which multiply in $\mathbb{F}_{2^8}$ and are not S-box circuits (they are the files `aes107_d13.slp` and `aes108_d12.slp`). A gist is editable by its owner and its date is GitHub's, so these are weak

timestamps: they indicate that the files existed on 2026-10-11, the same day as this preprint, and are not evidence of priority over earlier work. **Both roots contain the same seven S-box files:** 106/34, 106/24, 110/38, 113/18, 116/17, 121/16 and inverse 109/31 (Table 6); the two `SHA256SUMS` files differ only in those two multiplier lines. Both files also list multiplier circuits that are not in this pack. **The depth points** 112/18**,** 112/20**,** 111/21 **and inverse** 109/30 **are released, but are not under either commitment root.**

3 Method

3.1 The Boyar–Peralta layout

The circuits we start from have the following shape (Boyar and Peralta [5]; the block names are ours). Write $A \in \mathbb{F}_{2^8}$ as $a_0 b_0 + a_1 b_1$ over a basis (b_0, b_1) of $\mathbb{F}_{2^8}$ over $\mathbb{F}_{2^4}$. A *top* layer of XORs computes 18 linear forms of the input. *MUL1* multiplies pairs of forms (9 ANDs) and sums them linearly into the four bits of the norm $X = A^{17} \in \mathbb{F}_{2^4}$. The *inverter* computes X^{-1} (5 ANDs and about 16 XORs including its input sums). The *bottom* multiplies X^{-1} with the same 18 forms (18 ANDs) and a linear layer, which includes the affine map of the S-box, produces the eight outputs. All four BP-style circuits that we dissected have $9 + 5 + 18 = 32$ ANDs. Table 2 shows where the gates sit.

Circuit	Top	MUL1	Inverter	Bottom	Total
BP/Calik 113	23	$9 + 16$	$5 + 14$	$18 + 28$	113
our 110 (110/38)	20	$9 + 17$	$5 + 13$	$18 + 28$	110
107 (rep. $206, 190, 224, 0$)	17	$9 + 19$	$5 + 13$	$18 + 26$	107
106 (rep. $85, 130, 12, 1$)	19	$9 + 17$	$5 + 13$	$18 + 25$	106

Table 2: Gate allocation by block, from our dissection script; MUL1, inverter and bottom are given as AND+XOR. The 113's MUL1 contains 10 bilinear sums, 4 additions of the linear part of the norm, and 2 inverter sums that sit at level one. The inverter has 21 gates (5 AND, 11 XOR in the core, plus 5 sums). Walks gain mostly in the top and bottom layers.

3.2 Representation sweep

The Boyar–Peralta circuit fixes one representation of $\mathbb{F}_{2^8}$ and one set of sub-bases. That the choice matters is known (Canright's basis search; the 2040 scaling and Frobenius variants of Maximov and Ekdahl, Section 5.4 of [9]); our sweep is a wider version of it. The layout can be rebuilt from parameters $(\theta_0, \theta_1, t, g)$, where (θ_0, θ_1) is a dual basis of $\mathbb{F}_{2^8}$ over $\mathbb{F}_{2^4}$ and (t, g) (a scaling and a Frobenius power, the parameters of Maximov and Ekdahl's transformation) selects, by conjugating the 113's inverter, a set Λ of 9 symmetric product forms in the class of the 113's own set. That class (class B) holds 10 of the 25 admissible sets (Section 3.6); scaling and Frobenius keep the class, so the sweep cannot reach the other 15 sets. We show in our notes that every $\mathbb{F}_{2^4}$-basis supports the form sharing, not only normal bases, and that all sub-basis choices (normal or polynomial) are instances of the same parameters, because the construction uses only trace functionals. We checked these statements by enumeration and by assertions in the code (every build asserts the 256-entry table), not in Lean. The parameter space has $17 \times 240 \times 60 = 244{,}800$ builds. For each representation `rep_sweep.py` (i) builds the 18 top forms, (ii) solves the MUL1 sums and the bottom matrix on the truth tables, (iii)

reuses the 113's own inverter, re-targeted by conjugation so that its cost is 21 gates in every representation (Table 2 counts 2 of the 113's inverter sums in MUL1), and (iv) realizes each linear layer with Paar, BP and RNBP heuristics (`linear_opt.py`), fixing constants by XNOR flips. Our scorer is slightly pessimistic: it rebuilds the 113's own representation with top 23 and MUL1 sums 14 exactly, but its bottom layer costs 30 or 31 against BP's 28.

3.3 Walks

After a build we run a random walk over functionally equal circuits (`sbox.py walk`). Two moves keep the value of every gate except one: *resubstitution*, which re-expresses a gate over two other existing signals that are not its descendants (it keeps every gate's value), and *re-association*, $(a \oplus b) \oplus v \to (a \oplus v) \oplus b$ when the inner gate has fan-out one (the outer gate keeps its value; the inner gate changes from $a \oplus b$ to $a \oplus v$, which is not observable elsewhere because its fan-out is one). Dead gates stay as tombstones that resubstitution can reuse. Because every output and every gate with fan-out above one keeps its truth table, a walk can never change the top and bottom matrices, only how they are shared. That is why our walks within the 113's own representation plateaued at 110. A change of representation rewrites both matrices at once. We checked that this is not a symmetry of the cost: the inversion does not commute with the affine output map.

3.4 Window SAT

We also cut a window of 4 to 9 gates from a circuit and ask a SAT solver for a replacement with fewer gates computing the same outputs from the signals available before the window, with freedom in XNOR polarity. On the 113 this found 112 (one gate) and nothing more. It did not reach the representation-sweep results and we do not credit the 106 to it.

3.5 Depth lane

To trace the gate–depth trade-off we used the depth lane (`sbox_depth.py`). It rebalances XOR/XNOR trees to minimum depth at equal gate count, runs the walk with a penalty $\lambda \max(0, \text{depth} - \text{cap})$ for caps from 14 to 30, runs SAT windows whose splices must not exceed the cap, builds a depth-aware version of the representation sweep (`rep_depth.py`, which rebalances every build), and keeps the non-dominated (gates, depth) points. Unlike the gate-preserving walk of the previous subsections, its depth-aware moves can change the type of a gate, which is how the 32 ANDs of the 106/34 became the 34 ANDs of the 106/24. The points of Table 1 at depths 16 to 24 and the inverse circuits come from this lane; the inverse circuits come from running the same walk and window tools (`invtool.py`) on the inverse table.

The lane was seeded with published circuits and with our own; there are two distinct seed sets across the three lanes we ran (table in Section 4.3). The depth points share the representation of published circuits and are likely local improvements of them (Section 4.3), and we credit the published seeds. The driver logs (`provenance/`) give the Pareto front after most rounds (the first lane's log has no round 3 and the second lane's no round 5); they do not give the steps of each point, and the per-circuit logs are not saved. `provenance/front.md` is the first lane's front: it lists 113/18 and 112/20 but not 112/18, which first appears in the second lane.

3.6 Structural facts that bound the family

Computer-assisted exhaustive searches in our code, scoped to the BP-style family, not Lean-checked and not independently reproduced, show: (a) $\mathbb{F}_{2^4}$ inversion needs 5 ANDs in this setting (the inverter's 5 is not improvable by changing its AND count); (b) the norm needs at least 5 ANDs, and exactly 9 if every AND reads two of the 18 pooled top forms (no set of 8 or 7 candidate products spans the target; a local search outside the pool stalled at deficiency 2 to 3); (c) there are exactly 25 symmetric rank-9 form sets, in two classes up to scaling and Frobenius (class A has 15, class B has 10); the 113 uses class B and class A has not been built. Whether the S-box needs fewer than 32 ANDs is open for us. The NIST list [2] contains circuits with 28 ANDs (at larger total gate counts), so our 32 to 34 is not the smallest known AND count.

4 Results

4.1 Forward S-box

The sweep scored all 244,800 builds (60 shards, about 20 minutes). Under one fixed scorer the BP representation builds to 116 (115 at 300 restarts); in a first sample of 2,000 random representations (10 restarts), 75 scored 116 or better and the best built to 111. We walked the 288 builds with raw score at most 112 for 300 s each: 38 reach 106 gates and none goes lower. Deepening the 30 best walks (two seeds, 1800 s each) left all of them at 106. In the first sample, four representations walked as follows: $(85, 130, 12, 1)$ raw 112 to 106 (two seeds, both 106), $(206, 190, 224, 0)$ raw 111 to 107, $(66, 159, 1, 0)$ raw 113 to 109 and $(208, 222, 93, 0)$ raw 111 to 110; the raw score predicts the walked score with a gain of 1 to 6. This gives the 106-gate circuit with 32 AND, 71 XOR and 3 XNOR at depth 34 (Lean-checked). The depth lane produced the 106-gate circuit at depth 24, which uses 34 ANDs, 69 XORs and 3 XNORs (Lean-checked, Appendix A). It descends from the depth-34 circuit: 88 of its 106 gate functions occur in that circuit (Section 4.3).

At depth 16, our 121 gates are 4 gates (3.2%) below the 125 of the CMT page (depth 16) and one level deeper than the 125 of the NIST repository (depth 15); we do not call it a same-depth win. At depths 17, 18 and 21 we have 116, 112 and 111 gates, which are 9 to 14 gates below the smallest published circuit of depth at most that bound that we found, but each is two to six levels deeper than the NIST 125 (one to five than the CMT 125). Our 113/18 and 112/20 are dominated by the 112/18. For size alone, 106 is 4 gates (3.6%) below the NIST 110 at depth 22, and 7 below the BP 113 at depth 27; the depth-24 circuit is two levels deeper than the first and three levels shallower than the second.

4.2 Inverse S-box

The walk and window tools run on the inverse table give 109 gates at depth 31 $(32 + 70 + 7)$ and at depth 30 $(32 + 71 + 6)$, both Lean-checked; the second dominates the first. These are 3 gates below the NIST 112 at depth 26 and 12 below the CMT 121 at depth 21, and are 4 to 5 and 9 to 10 levels deeper respectively. We have no inverse circuit at depth 26 or less and make no claim there. Maximov–Ekdahl's 101-gate (plus one NOT gate) inverse is in the richer gate set.

4.3 Provenance and lineage

We compared the gates of our circuits with the prior circuits we hold, by function: a gate matches if its 256-bit truth table, up to complement, occurs among the gates of the other circuit (`tools/overlap.py`). Table 3 gives the counts. The two NIST 110/22 circuits were loaded as seeds into the later lanes of the depth lane's search (table below); one of them (the file `aes-sbox-a32-ad5-g110-gd22`) is gate-for-gate identical to a 110-gate depth-22 circuit that we also held, so **that circuit is NIST's and is not counted as ours**, and it is not in the pack.

Circuit	NIST a32	NIST a34	CMT 125	NIST 125	BP/Calik	our 106/34
121 / 16	77 (64%)	77 (64%)	104 (86%)	90 (74%)	77 (64%)	17 (14%)
116 / 17	84 (72%)	83 (72%)	90 (78%)	86 (74%)	79 (68%)	17 (15%)
113 / 18	77 (68%)	80 (71%)	81 (72%)	76 (67%)	82 (73%)	17 (15%)
112 / 18	76 (68%)	79 (71%)	77 (69%)	72 (64%)	81 (72%)	17 (15%)
112 / 20	84 (75%)	84 (75%)	77 (69%)	74 (66%)	80 (71%)	18 (16%)
111 / 21	89 (80%)	92 (83%)	79 (71%)	74 (67%)	79 (71%)	17 (15%)
106 / 24	18 (17%)	18 (17%)	18 (17%)	17 (16%)	17 (16%)	88 (83%)
110 / 38	76 (69%)	74 (67%)	77 (70%)	71 (65%)	88 (80%)	18 (16%)

Table 3: Number of gates of the row circuit whose function occurs in the column circuit. Row totals are the gate counts. The 106/34 column is our sweep circuit.

Baseline. Any two circuits built in the same field representation share many gate functions, so Table 3 needs a baseline. Table 4 gives the pairwise overlaps among the prior circuits (NIST a32, NIST a34, NIST 125, CMT 125, BP/Calik 113) and our 110/38 (`tools/overlap.py`). Among the prior circuits the overlaps are 56% to 86% (56% to 74% except for the two NIST 110/22 circuits against each other); our 110/38, a walk from the BP/Calik 113, shares 65% to 80% with them. The depth points at depths 16 to 21 share 64% to 86% of their gate

Circuit	NIST a32	NIST a34	NIST 125	CMT 125	BP/Calik	our 110/38
NIST a32 (110)	–	95 (86%)	70 (64%)	75 (68%)	81 (74%)	76 (69%)
NIST a34 (110)	95 (86%)	–	71 (65%)	77 (70%)	80 (73%)	74 (67%)
NIST 125	70 (56%)	71 (57%)	–	88 (70%)	75 (60%)	71 (57%)
CMT 125	75 (60%)	77 (62%)	88 (70%)	–	84 (67%)	77 (62%)
BP/Calik 113	81 (72%)	80 (71%)	75 (66%)	84 (74%)	–	88 (78%)
our 110/38	76 (69%)	74 (67%)	71 (65%)	77 (70%)	88 (80%)	–

Table 4: Baseline: gates of the row circuit whose function occurs in the column circuit, among prior circuits and our 110/38.

functions with the published circuits. Among prior circuits that are not siblings the overlaps are 56% to 74%; only the two NIST 110/22 circuits, which are siblings, reach 86%. Two of our points sit at the level of that sibling pair: the 121/16 shares 104 of 121 (86%) with the CMT 125/16, and the 111/21 shares 92 of 111 (83%) with NIST a34 (110/22). We read them as derived from those circuits: the 121/16 is most likely a −4-gate local improvement of the CMT 125/16 (the Boyar–Find–Peralta circuit [7]), and the 111/21 is most likely a +1-gate, −1-level rebalancing of NIST a34. The other points are weaker: 90 of 116 (78%, CMT 125) for the 116/17, 84 of 112 (75%, NIST a32 and a34) for the 112/20, 82 of 113 (73%, BP/Calik

113) for the 113/18 and 81 of 112 (72%, BP/Calik 113) for the 112/18; these are at or just above the non-sibling range, they show the same representation as published circuits, and they do not identify a parent. We credit the NIST, CMT and Boyar–Peralta–Calik circuits as likely seeds. The cell-level value is only a measure of shared intermediate functions, not proof of descent.

Inverse circuits. Table 5 gives the same measure for the inverse circuits. The two prior inverse circuits, the NIST 112/26 and the CMT 121/21, share 77 of 121 (64%; 77 of 112, 69% the other way). Our inverse 109/30 and 109/31 share only 24 of 109 (22%) with the NIST inverse and 25 of 109 (23%) with the CMT inverse, far below the 64% to 69% baseline between the two prior inverse circuits. So they are not local edits of either; their parent is unknown (the inverse circuits come from the walk and window tools run on the inverse table, with no retained output files). The two of ours share 35 of 109 (32%) with each other, so they are not close relatives either.

Circuit	NIST inv 112	CMT inv 121	our 109/30	our 109/31
our 109/30	24 (22%)	25 (23%)	–	35 (32%)
our 109/31	24 (22%)	25 (23%)	35 (32%)	–
NIST inv 112	–	77 (69%)	24 (21%)	24 (21%)
CMT inv 121	77 (64%)	–	26 (21%)	26 (21%)

Table 5: Inverse circuits: gates of the row circuit whose function occurs in the column circuit.

Lanes and seeds. We ran the depth lane in three lanes (driver logs in `provenance/`). We list what we can recover from the driver script and the file dates in our working tree, not from run records. All lanes also read the Pareto fronts of the earlier lanes when they started.

Lane (log)	Seed set	Released files first logged or found in it
First (`driver.log`, 10 rounds)	CMT 125, BP/Calik 113, our 110/38, later our sweep walks; the NIST folder is dated after this lane began, so probably no NIST seeds	113/18 (round 1); 121/16, 112/20, 111/21 (round 4); 116/17 (round 5); 106/24 (round 10). Only the points are logged
Second (`driver_n.log`, prefix `n`)	the above, the two NIST 110/22 circuits, and the first lane's front	112/18 (round 6; output files with prefix `n` only), 113/18 output files
Third (`driver_r.log`)	as the second lane	output files with prefix `r`, a prefix the first lane also used; so for the 116/17, 111/21 and 112/20 output files the lane is unknown

The inverse circuits and the 121/16 have no retained output file, so their lane and seed set are unknown. For every file the exact parent is unknown. We do not infer the seed set of a lane from the absence of a point in its log. The first lane logged a 111/21 point (round 4), and that lane probably had no NIST seeds, but the released 111/21 is closest to NIST a34 (83%), which points to a later lane; the logged point and the released file may be different circuits, and we do not claim they are the same.

The 106/24 is different. It shares 88 of 106 gate functions (83%) with our sweep circuit 106/34 and only 18 (17%) with each of the two NIST 110/22 circuits (and 17 to 18 with the

CMT 125, NIST 125 and BP/Calik 113). It is far below the 56% to 86% that prior circuits share with each other and close to our own 106/34, which is evidence that it descends from the sweep circuit through the depth lane, whose moves can change gate types (32 ANDs became 34); it is not proof, since we did not save the per-circuit run logs.

5 Negative results and what did not work (within our method)

- **Walks in one representation stop at** 110**.** All chains, several rounds, plateaued at 110 ($32 + 73 + 5$, depth 38); window SAT took the 113 to 112 only.
- **Walks after the sweep stop at** 106**.** 288 builds walked, 38 at 106, none lower; 60 deepened walks stop at 106. Within the BP-layout family and this walk, 106 is the best we found.
- **Tower rebuild.** Rebuilding the S-box over $\mathbb{F}_{2^4}$ with our 30-gate $\mathbb{F}_{2^4}$ multiplier (three multiplications and an inverter) gives 164 gates, far above 113; used only as an alternative start.
- **XNOR polarity absorption of the norm's linear part** (the AND/XOR analogue of the NOR trick of Maximov and Ekdahl) gives 119 to 123 against 113 for the same representation: of the 26 buildable polarity patterns none helps, as the bottom layer grows from 27 to 35 while the top shrinks only from 24 to 23. Not rerun.
- **GF(2^8) multiplier chains** (a second search lane) were stopped at 107 with no hit below.
- **AND count.** Nothing below 32 ANDs in this family (Section 3).
- **Depth.** We have no forward circuit below depth 16 and no inverse circuit below depth 30; at depth 16 the best we found is 121, against a published 125 at depth 15.
- **Untested.** The class-A form sets, the exact-synthesis inverter and the quadratic norm with fewer than 9 products were planned and not run, so we say nothing about them.

6 What the numbers mean in practice

Fewer gates saves area in a hardware implementation and instructions in a bitsliced software implementation (one word-wide instruction per gate), before technology-specific effects. We did not synthesize or benchmark anything, so we give no area, time or energy figures, and the cost of XNOR relative to XOR depends on the library. Our 106 and 109 circuits are deeper than the circuits they beat in size, which matters when latency counts; for that, the depth-budget rows of Table 1 are the better match, within the {AND, XOR, XNOR} model. The AND count (32 to 34) is about that of earlier work (the 113 has 32, the 125 has 34) and the NIST list has circuits with 28 ANDs, so there is no improvement for masked implementations, where cost grows with ANDs and not with XORs.

7 Artifacts

The pack `repro/`, published at `https://github.com/Swapnil-jain/gf2m-circuits`, contains the circuits (JSON, and a readable listing per circuit), the Lean files and outputs, the

generator `gen_sbox_lean.py`, the verifier, the commitment files and a README with exact commands. Circuit JSON: `{"gates": [[op, x, y], ...], "outputs": [...]}`, signals 0 to 7 are $U_0, \ldots, U_7$ (MSB first) and signal $8 + i$ is gate i. The listing format is one gate per line, `T`i `= X + Y` (XOR), `x` (AND), or `~` (XNOR). Running the verifier on every circuit gives: all 11 files pass the exhaustive and symbolic checks, with all seven controls rejected (`verify_results.txt`). The pack also holds the prior-work circuits we quote as data (`prior/`, raw files and our JSON conversion), the parser and depth recount `tools/prior_circuits.py` (output in `prior_recount.txt`), the gate-function overlap script `tools/overlap.py` (output in `overlap_results.txt`), the depth script `tools/and_depth.py` and the driver logs of the depth lane (`provenance/`).

File	Circuit	Lean	Commitment	SHA-256 (first 16 hex)
`sbox106_d24`	fwd 106 / 24 $(34 + 69 + 3)$	L	both roots	`b189eca69c8e5565`
`sbox106_d34`	fwd 106 / 34 $(32 + 71 + 3)$	L	both roots	`dc9c01a32842ac02`
`sbox110_d38`	fwd 110 / 38 $(32 + 73 + 5)$	L	both roots	`bcb9239523918a60`
`sbox113_d18`	fwd 113 / 18 $(34 + 76 + 3)$	L	both roots	`81f32010dc70320f`
`sbox116_d17`	fwd 116 / 17 $(34 + 78 + 4)$	L	both roots	`342739572efee6fc`
`sbox121_d16`	fwd 121 / 16 $(34 + 83 + 4)$	L	both roots	`cf7ae66686037588`
`invsbox109_d31`	inv 109 / 31 $(32 + 70 + 7)$	L	both roots	`d97216648dfac94a`
Released, not under the commitment root (not in `SHA256SUMS`):				
`sbox111_d21`	fwd 111 / 21 $(34 + 74 + 3)$	L	none	`edb16a1872c86d79`
`sbox112_d18`	fwd 112 / 18 $(34 + 75 + 3)$	L	none	`7c4ab111baa7ae9c`
`sbox112_d20`	fwd 112 / 20 $(34 + 75 + 3)$	L	none	`03d7c644009cc979`
`invsbox109_d30`	inv 109 / 30 $(32 + 71 + 6)$	L	none	`0c0fdb0a17bf1efe`

Table 6: Circuit files (each is a `.json` file in `circuits/`). In the commitment file the first six forward files are named `sbox_106.json` (106/34), `sbox_106_d24.json`, `sbox_110.json` (110/38), `sbox_113_d18.json`, `sbox_116_d17.json`, `sbox_121_d16.json` and the inverse `invsbox_109_d31.json`; the hashes are those of the committed files (`commitment/NAMEMAP.txt`). Lean files and outputs (all with `.out`): `Sbox106`, `Sbox106d24`, `Sbox110`, `Sbox113d18`, `Sbox116d17`, `Sbox121d16`, `InvSbox109`, `Sbox111d21`, `Sbox112d18`, `Sbox112d20`, `InvSbox109d30`.

8 Conclusion

Rebuilding the Boyar–Peralta layout in other field representations before walking took our 110-gate plateau to 106, which is 4 gates below the 110-gate depth-22 circuit on the NIST page, at two more levels of depth. The depth lane, seeded with published circuits, gives S-box circuits of 121 to 111 gates at depths 16 to 21 against 125 at depth 15 or 16 in the published lists we read, and a 109-gate inverse. The smallest circuits are deeper than the circuits they beat in size, the AND count (32 to 34) is about that of earlier work, and richer gate sets reach 102 (so the depth-budget comparisons hold only in our gate model). We make no optimality claim: the family we search, the BP layout with 32 to 34 ANDs, stopped at 106 in all of our walks, and whether a different family (class-A form sets, fewer ANDs, other architectures) goes lower is open.

Acknowledgements. Computational search and drafting were assisted by AI tools; all released circuits were machine-verified.

A The 106-gate circuit (depth 24)

The circuit `sbox106_d24` (34 AND, 69 XOR, 3 XNOR). Inputs `U0..U7` (`U0` the MSB), "+" is XOR, "x" is AND and "xn" is XNOR (in the pack's `.txt` listings XNOR is written with a tilde); outputs `S0..S7` with `S0` the MSB. It passes the exhaustive and symbolic checks, all controls and a Lean kernel proof of functional correctness.

```
T0 = U2 + U3
T1 = U4 + U5
T2 = U2 + U5
T3 = U6 + T2
T4 = U1 + T3
T5 = U7 + T3
T6 = T1 + T4
T7 = T0 + T6
T8 = T5 + T7
T9 = U4 + T8
T10 = U1 x T8
T11 = U0 + T8
T12 = T6 + T11
T13 = T3 x T12
T14 = U0 + T7
T15 = U3 + T11
T16 = T6 x T15
T17 = T9 x T14
T18 = T2 + T14
T19 = T7 x T18
T20 = T15 + T18
T21 = T17 + T19
T22 = T21 + U4
T23 = T0 x T20
T24 = T10 + U7
T25 = T24 + T17
T26 = U1 + T18
T27 = T5 x T26
T28 = T13 + T25
T29 = T8 + T12
T30 = T4 x T29
T31 = T9 + T26
T32 = T11 x T31
T33 = T1 + T25
T34 = T23 + T32
T35 = T34 + T22
T36 = T29 + T27
T37 = T28 + T36
T38 = T16 + T31
T39 = T36 + T38
T40 = T39 + T22
T41 = T37 x T40
T42 = T34 + T39
T43 = T33 + T32
T44 = T43 + T30
T45 = T44 x T42
T46 = T41 + T45
T47 = T42 x T46
T48 = T35 x T46
T49 = T40 + T48
T50 = T35 + T47
T51 = T50 x T4
T52 = T50 x T29
T53 = T49 x U1
T54 = T8 x T49
T55 = T49 + T50
T56 = T3 x T55
T57 = T12 x T55
T58 = T53 + T56
T59 = T44 + T37
T60 = T35 x T59
T61 = T41 + T60
T62 = T37 x T61
T63 = T59 x T61
T64 = T59 + T62
T65 = T44 + T63
T66 = T65 x T20
T67 = T65 x T0
T68 = T65 + T64
T69 = T64 x T15
T70 = T6 x T64
T71 = T68 x T18
T72 = T7 x T68
T73 = T66 + T69
T74 = T67 + T70
T75 = T55 + T64
T76 = T65 + T50
T77 = T31 x T75
T78 = T11 x T75
T79 = T9 x T76
T80 = T14 x T76
T81 = T74 + T78
T82 = T73 + T56
T83 = T82 + T51
T84 = T74 + T52
T85 = T84 + T57
T86 = T67 + T72
T87 = T86 + T83
T88 = T73 + T77
T89 = T78 + T87
T90 = T89 xn T80
T91 = T52 + T87
T92 = T91 + T54
T93 = T69 + T71
T94 = T75 + T76
T95 = T26 x T94
T96 = T5 x T94
T97 = T96 xn T81
T98 = T83 + T97
T99 = T95 + T88
T100 = T93 + T99
T101 = T98 + T100
T102 = T95 + T85
T103 = T102 + T79
T104 = T103 + T101
T105 = T58 xn T101
S0 = T105
S1 = T104
S2 = T98
S3 = T92
S4 = T99
S5 = T85
S6 = T97
S7 = T90
```

B Reproducing the numbers

Run from `repro/`:

- `for f in circuits/*.json; do python3 -I tools/verify_sbox.py $f; done`
- `lean lean/Sbox106d24.lean` for a proof
- `shasum -a 256 commitment/SHA256SUMS` to compare with the root (`SHA256SUMS.v1` for the first root)
- `python3 -I tools/prior_circuits.py` recounts the prior circuits

- `python3 -I tools/overlap.py` reproduces Tables 3, 4 and 5
- `python3 -I tools/and_depth.py` gives gate depth and AND depth of every circuit

The verifier takes a few seconds per circuit. Only verification, the recounts, the overlaps and the Lean checks are reproducible from the pack; the search code (`sbox.py`, `sbox_depth.py`, `rep_depth.py`, `invtool.py`, `linear_opt.py`) is not released, so none of the circuits can be regenerated from the pack. The sweep statistics, Table 2, the structural facts (a) to (c), the second checkers and the numeric encoding of the representation tuples (which is that of the sweep code and is not defined here) are not reproducible from the pack. The pack includes `rep_sweep.py` for reference only, because it imports search code that is not in the pack. The names `tools/sbox.py check` and `ideas/sbox/indep_check.py` in Section 2.2 refer to our working tree, not to the pack.

References

[1] J. Boyar, M. Dworkin, R. Peralta, M. Turan, C. Calik, L. Brandao (current members of the Circuit Minimization Team, CMT; past collaborators listed on the page include M. Bartock, R. Collazo, M. Find, M. Fischer, M. Cenk, C. Wood, A. Visconti, C. Schiavo, H. Gao, B. Strackbein, L. Bassham), *Circuit Minimization Work*, `https://www.cs.yale.edu/homes/peralta/CircuitStuff/CMT.html`.

[2] NIST Computer Security Resource Center, *Circuit Complexity*, "List of circuits" (AES S-box circuits over {AND, XOR, XNOR}, tables added in 2026), `https://csrc.nist.gov/projects/circuit-complexity/list-of-circuits`; the page showed "last updated October 6, 2026" when we read it on 2026-10-11.

[3] NIST, `usnistgov/Circuits`, directories `data/slp/aes/aes-sbox` and `data/slp/aes/aes-invsbox`, `https://github.com/usnistgov/Circuits` (file names read via the GitHub contents listing on 2026-10-11; the names encode AND count, AND depth, gate count and gate depth; the circuits we quote were parsed and recounted by us, see Section 1.1).

[4] J. Boyar and R. Peralta, A new combinational logic minimization technique with applications to cryptology, in *Experimental Algorithms (SEA 2010)*, LNCS 6049, Springer, 2010, pp. 178–189, doi:10.1007/978-3-642-13193-6_16.

[5] J. Boyar, P. Matthews, R. Peralta, Logic minimization techniques with applications to cryptology, *Journal of Cryptology* 26(2), 2013, pp. 280–312, doi:10.1007/s00145-012-9124-7.

[6] J. Boyar and R. Peralta, A small depth-16 circuit for the AES S-box, in *Information Security and Privacy Research (SEC 2012)*, IFIP Advances in Information and Communication Technology vol. 376 (D. Gritzalis, S. Furnell, M. Theoharidou, eds.), Springer, 2012, pp. 287–298, doi:10.1007/978-3-642-30436-1_24; ePrint 2011/332.

[7] J. Boyar, M. G. Find, R. Peralta, Small low-depth circuits for cryptographic applications, *Cryptography and Communications* 11(1), 2019, pp. 109–127, doi:10.1007/s12095-018-0296-3 (published online 2018).

[8] D. Canright, A very compact S-box for AES, in *Cryptographic Hardware and Embedded Systems (CHES 2005)*, LNCS 3659, Springer, 2005, pp. 441–455, doi:10.1007/11545262_32.

[9] A. Maximov and P. Ekdahl, New circuit minimization techniques for smaller and faster AES SBoxes, *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2019(4), pp. 91–125, doi:10.13154/tches.v2019.i4.91-125.

[10] A. Reyhani-Masoleh, M. Taha, D. Ashmawy, Smashing the implementation records of AES S-box, *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2018(2), pp. 298–336, doi:10.13154/tches.v2018.i2.298-336.

[11] R. Ueno, N. Homma, Y. Sugawara, Y. Nogami, T. Aoki, Highly efficient GF(2^8) inversion circuit based on redundant GF arithmetic and its application to AES design, in *Cryptographic Hardware and Embedded Systems (CHES 2015)*, LNCS 9293, Springer, 2015, pp. 63–80, doi:10.1007/978-3-662-48324-4_4.

[12] R. Ueno, N. Homma, Y. Nogami, T. Aoki, Highly efficient GF(2^8) inversion circuit based on hybrid GF representations, *Journal of Cryptographic Engineering* 9(2), 2019, pp. 101–113, doi:10.1007/s13389-018-0187-8 (bibliographic data checked on Crossref).

[13] G. Kim, S. Baek, Y. Jeon, V. Hadžić, J. Kim, Bitsliced segment-based search technique for low-depth and hardware-efficient S-box circuits, *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2026(3), pp. 1152–1176, doi:10.46586/tches.v2026.i3.1152-1176; ePrint 2026/726; artifact at `https://artifacts.iacr.org/tches/2026/a37` (circuit sizes read from the artifact README).

[14] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS 197, 2001 (updated 2023), doi:10.6028/NIST.FIPS.197-upd1.

[15] C. Paar, Optimized arithmetic for Reed–Solomon encoders, in *Proc. IEEE International Symposium on Information Theory (ISIT)*, 1997, p. 250, doi:10.1109/ISIT.1997.613165; and Ph.D. thesis, "Effiziente VLSI-Architekturen für bit-parallele Arithmetik in endlichen Körpern", Universität Essen (now Duisburg-Essen), 1994.

[16] A. Visconti, C. V. Schiavo, R. Peralta, Improved upper bounds for the expected circuit complexity of dense systems of linear equations over GF(2), *Information Processing Letters* 137, 2018, pp. 1–5, doi:10.1016/j.ipl.2018.04.010.